

SKILL PROBLEMS

5(a): Single Quotes

code

```
GNU nano 8.7.1
#include <stdio.h>
#include <string.h>

int main() {
    char input[100];

    printf("Enter text inside single quotes: ");
    fgets(input, sizeof(input), stdin);

    printf("Literal content: %s", input);

    return 0;
}|
```

Output

```
spoorthi@LAPTOP:~$ gcc singlequotes.c -o singlequotes
spoorthi@LAPTOP:~$ ./singlequotes
Enter text inside single quotes: Hello World $Home
Literal content: Hello World $Home
spoorthi@LAPTOP:~$ |
```

5(b): Double Quotes

Code

```
#include <stdio.h>
#include <string.h>

int main() {
    char name[50];

    printf("Enter your name: ");
    fgets(name, sizeof(name), stdin);

    printf("Double quoted value: \"%s\"", name);

    return 0;
}|
```

Output

```
spoorthi@LAPTOP:~$ nano doublequotes.c
spoorthi@LAPTOP:~$ gcc doublequotes.c -o doublequotes
spoorthi@LAPTOP:~$ ./doublequotes
Enter your name: Hello Spoorthi
Double quoted value: "Hello Spoorthi"
spoorthi@LAPTOP:~$ |
```

6(a) — Process Escape Sequences

Code

```
#include <stdio.h>

int main() {
    char input[200];

    printf("Enter text: ");
    fgets(input, sizeof(input), stdin);

    printf("Parsed input: %s", input);

    return 0;
}|
```

Output

```
spoorthi@LAPTOP:~$ gcc escape_sequences.c -o escape_sequences
spoorthi@LAPTOP:~$ ./escape_sequences
Enter text: Hello Linux
Parsed input: Hello Linux
spoorthi@LAPTOP:~$ |
```

6(b) — Child Process

Code

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid;

    pid = fork();

    if (pid < 0) {
        printf("Fork failed\n");
    }
    else if (pid == 0) {
        printf("Child process created\n");
        printf("Child process executing ls command\n");

        execlp("ls", "ls", "-l", NULL);

        printf("Execution failed\n");
    }
    else {
        wait(NULL);
        printf("Parent process completed\n");
    }

    return 0;
}
```

Output

```

spoorthi@LAPTOP:~$ nano child_process.c
spoorthi@LAPTOP:~$ gcc child_process.c -o child_process
spoorthi@LAPTOP:~$ ./child_process
Child process created
Child process executing ls command
total 1416
-rw-r--r-- 1 spoorthi spoorthi    10 Jul 29 08:22 1.txt
-rw-r--r-- 1 spoorthi spoorthi    10 Jul 29 06:54 1.txt.save
-rw-r--r-- 1 spoorthi spoorthi     9 Jul 29 06:59 12.txt
-rw-r--r-- 1 spoorthi spoorthi    32 Jul 29 07:09 3.txt
-rw-r--r-- 1 spoorthi spoorthi    15 Jul 29 08:23 4.txt
drwxr-xr-x 2 spoorthi spoorthi  4096 Aug 22 05:36 SKILL
drwxr-xr-x 2 spoorthi spoorthi  4096 Aug 22 05:30 Skill
-rwxr-xr-x 1 spoorthi spoorthi 16080 Aug 12 17:42 child
-rw-r--r-- 1 spoorthi spoorthi   335 Aug 22 04:42 child.c
-rw-r--r-- 1 spoorthi spoorthi   186 Aug 22 04:43 child1.c
-rwxr-xr-x 1 spoorthi spoorthi 16040 Aug 12 17:44 child2
-rw-r--r-- 1 spoorthi spoorthi   236 Aug 12 17:43 child2.c
-rwxr-xr-x 1 spoorthi spoorthi 16088 Sep 26 05:17 child_process
-rw-r--r-- 1 spoorthi spoorthi   478 Sep 26 05:17 child_process.c
-rwxr-xr-x 1 spoorthi spoorthi 16216 Aug 10 10:51 client
-rw-r--r-- 1 spoorthi spoorthi   264 Aug 10 10:50 client.c
-rwxr-xr-x 1 spoorthi spoorthi 16344 Sep  3 03:57 client1
-rw-r--r-- 1 spoorthi spoorthi   674 Sep  3 03:57 client1.c
-rwxr-xr-x 1 spoorthi spoorthi 16032 Aug 10 07:20 cow
-rw-r--r-- 1 spoorthi spoorthi   237 Aug 10 07:20 cow.c
-rwxr-xr-x 1 spoorthi spoorthi 16464 Aug 10 07:50 deadlock_prevention
-rw-r--r-- 1 spoorthi spoorthi   854 Aug 10 07:49 deadlock_prevention.c
-rw-r--r-- 1 spoorthi spoorthi     0 Aug 10 07:22 dest.txt
-rwxr-xr-x 1 spoorthi spoorthi 16096 Sep 26 05:10 doublequotes
-rw-r--r-- 1 spoorthi spoorthi   211 Sep 26 05:09 doublequotes.c
-rwxr-xr-x 1 spoorthi spoorthi 16080 Aug 10 07:23 dup2_demo
-rw-r--r-- 1 spoorthi spoorthi   234 Aug 10 07:23 dup2_demo.c
-rwxr-xr-x 1 spoorthi spoorthi 16104 Sep 26 05:15 escape_sequences
-rw-r--r-- 1 spoorthi spoorthi   180 Sep 26 05:13 escape_sequences.c
-rwxr-xr-x 1 spoorthi spoorthi 15992 Aug 12 17:44 exec
-rw-r--r-- 1 spoorthi spoorthi   179 Aug 12 17:44 exec.c
-rwxr-xr-x 1 spoorthi spoorthi 16096 Sep  2 05:33 fcfs
-rw-r--r-- 1 spoorthi spoorthi   473 Sep  2 05:33 fcfs.c
-rwxr-xr-x 1 spoorthi spoorthi 16072 Aug 26 05:22 fd

```

```

-rwxr-xr-x 1 spoorthi spoorthi 16176 Aug 10 10:51 server
-rw-r--r-- 1 spoorthi spoorthi 292 Aug 10 10:50 server.c
-rwxr-xr-x 1 spoorthi spoorthi 16160 Sep 2 06:05 sigchild
-rw-r--r-- 1 spoorthi spoorthi 432 Sep 2 06:05 sigchild.c
-rwxr-xr-x 1 spoorthi spoorthi 16072 Aug 10 11:00 signal
-rw-r--r-- 1 spoorthi spoorthi 278 Aug 10 11:00 signal.c
-rwxr-xr-x 1 spoorthi spoorthi 16112 Sep 2 06:00 signall
-rw-r--r-- 1 spoorthi spoorthi 262 Sep 2 06:00 signall.c
-rwxr-xr-x 1 spoorthi spoorthi 16256 Sep 2 06:07 sigpipe
-rw-r--r-- 1 spoorthi spoorthi 459 Sep 2 06:07 sigpipe.c
-rwxr-xr-x 1 spoorthi spoorthi 16080 Sep 2 06:09 sigsegv
-rw-r--r-- 1 spoorthi spoorthi 371 Sep 2 06:09 sigsegv.c
-rwxr-xr-x 1 spoorthi spoorthi 16160 Sep 2 06:03 sigterm
-rw-r--r-- 1 spoorthi spoorthi 285 Sep 2 06:03 sigterm.c
-rwxr-xr-x 1 spoorthi spoorthi 16192 Aug 22 05:18 simplecommandloop
-rw-r--r-- 1 spoorthi spoorthi 302 Aug 22 05:17 simplecommandloop.c
-rw-r--r-- 1 spoorthi spoorthi 224 Sep 26 05:03 singleQuotes.c
-rwxr-xr-x 1 spoorthi spoorthi 16096 Sep 26 05:06 singlequotes
-rw-r--r-- 1 spoorthi spoorthi 224 Sep 26 05:05 singlequotes.c
-rwxr-xr-x 1 spoorthi spoorthi 16096 Sep 2 05:37 sjf
-rw-r--r-- 1 spoorthi spoorthi 572 Sep 2 05:37 sjf.c
-rwxr-xr-x 1 spoorthi spoorthi 16032 Aug 22 04:47 sleep
-rw-r--r-- 1 spoorthi spoorthi 254 Aug 12 17:49 sleep.c
-rwxr-xr-x 1 spoorthi spoorthi 16176 Sep 3 03:56 socket
-rw-r--r-- 1 spoorthi spoorthi 480 Sep 3 03:56 socket.c
lrwxrwxrwx 1 spoorthi spoorthi 9 Aug 10 07:25 softlink.txt -> file1.txt
-rw-r--r-- 1 spoorthi spoorthi 439 Aug 6 04:17 source.c
-rwxr-xr-x 1 spoorthi spoorthi 16288 Sep 19 05:11 spoorthi_open
-rw-r--r-- 1 spoorthi spoorthi 568 Sep 19 05:11 spoorthi_open.c
-rwxr-xr-x 1 spoorthi spoorthi 16184 Sep 19 05:14 sudesrna_fd
-rw-r--r-- 1 spoorthi spoorthi 1437 Sep 19 05:13 sudesrna_fd.c
-rw-r--r-- 1 spoorthi spoorthi 12 Aug 26 05:21 test.txt
-rwxr-xr-x 1 spoorthi spoorthi 16032 Aug 1 06:05 wait
-rw-r--r-- 1 spoorthi spoorthi 211 Aug 1 06:05 wait.c
-rw-r--r-- 1 spoorthi spoorthi 211 Aug 1 06:04 wait.c.save
-rwxr-xr-x 1 spoorthi spoorthi 16040 Aug 10 06:54 zombie
-rw-r--r-- 1 spoorthi spoorthi 212 Aug 10 06:53 zombie.c
-rw-r--r-- 1 spoorthi spoorthi 212 Aug 10 06:53 zombie.c.save
Parent process completed
spoorthi@LAPTOP:~$

```

7(a) Process Synchronization Using waitpid()

Code

```

#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid;

    pid = fork();

    if (pid < 0) {
        printf("Fork failed\n");
        return 1;
    }

    if (pid == 0) {
        printf("Child process started\n");
        sleep(2);
        printf("Child process completed\n");
    }
    else {
        waitpid(pid, NULL, 0);
        printf("Parent detected child completion\n");
    }

    return 0;
}

```

Output

```

Parent process completed
spoorthi@LAPTOP:~$ nano waitpid_demo.c
spoorthi@LAPTOP:~$ gcc waitpid_demo.c -o waitpid_demo
spoorthi@LAPTOP:~$ ./waitpid_demo
Child process started
Child process completed
Parent detected child completion
spoorthi@LAPTOP:~$ |

```

7(b) — Retrieve PATH and Locate Executable

Code

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

int main() {
    char *path;
    char *copy;
    char *dir;

    path = getenv("PATH");

    if (path == NULL) {
        printf("PATH variable not found\n");
        return 1;
    }

    printf("Searching for ls command...\n");

    copy = strdup(path);
    dir = strtok(copy, ":");

    while (dir != NULL) {
        char fullpath[500];

        snprintf(fullpath, sizeof(fullpath), "%s/%s", dir, "ls");

        if (access(fullpath, X_OK) == 0) {
            printf("Executable found: %s\n", fullpath);
            free(copy);
            return 0;
        }

        dir = strtok(NULL, ":");
    }

    printf("Command not found\n");

    free(copy);

    return 0;
}

```

Output

```

Parent detected child completion
spoorthi@LAPTOP:~$ nano path_search.c
spoorthi@LAPTOP:~$ gcc path_search.c -o path_search
spoorthi@LAPTOP:~$ ./path_search
Searching for ls command...
Executable found: /usr/bin/ls
spoorthi@LAPTOP:~$ |

```

8(a) — Variable Expansion

Code

```

#include <stdio.h>
#include <stdlib.h>

int main() {
    char variable[50];
    char *value;

    printf("Enter variable name: ");
    scanf("%49s", variable);

    value = getenv(variable);

    if (value != NULL)
        printf("Expanded value: %s\n", value);
    else
        printf("Variable is undefined\n");

    return 0;
}

```

Output

```

spoorthi@LAPTOP:~$ nano variable_expand.c
spoorthi@LAPTOP:~$ gcc variable_expand.c -o variable_expand
spoorthi@LAPTOP:~$ ./variable_expand
Enter variable name: Home
Variable is undefined
spoorthi@LAPTOP:~$ ./variable_expand
Enter variable name: HOME
Expanded value: /home/spoorthi
spoorthi@LAPTOP:~$ |

```

8(b) — Built-in Command Dispatch

Code

```

#include <stdio.h>
#include <string.h>

void pwd_command() {
    printf("PWD command executed\n");
}

void cd_command() {
    printf("CD command executed\n");
}

void exit_command() {
    printf("EXIT command executed\n");
}

int main() {
    char command[50];

    printf("Enter command: ");
    scanf("%49s", command);

    if (strcmp(command, "pwd") == 0)
        pwd_command();
    else if (strcmp(command, "cd") == 0)
        cd_command();
    else if (strcmp(command, "exit") == 0)
        exit_command();
    else
        printf("Invalid command\n");

    return 0;
}

```

Output

```

C:/Users/jampa/AppData/Roaming/npm/.7/minic/C:/Users/jampa/AppData/Local/Py
spoorthi@LAPTOP:~$ nano builtin_dispatch.c
spoorthi@LAPTOP:~$ gcc builtin_dispatch.c -o builtin_dispatch
spoorthi@LAPTOP:~$ ./builtin_dispatch
Enter command: pwd
PWD command executed
spoorthi@LAPTOP:~$ ./builtin_dispatch
Enter command: exit
EXIT command executed
spoorthi@LAPTOP:~$ |

```

9(a) Change Directory

Code

```
#include <stdio.h>
#include <unistd.h>
#include <limits.h>

int main() {
    char path[PATH_MAX];

    printf("Enter directory path: ");
    scanf("%s", path);

    if (chdir(path) == 0) {
        printf("Directory changed successfully\n");
    } else {
        perror("Directory change failed");
    }

    return 0;
}
```

Output

```
spoorthi@LAPTOP:~$ nano change_directory.c
spoorthi@LAPTOP:~$ gcc change_directory.c -o change_directory
spoorthi@LAPTOP:~$ ./change_directory
Enter directory path: /tmp
Directory changed successfully
spoorthi@LAPTOP:~$ |
```

9(b) — Built-in Command Dispatch

Code

```

#include <stdio.h>
#include <string.h>

void execute_command(char command[]) {
    if (strcmp(command, "pwd") == 0) {
        printf("Executing PWD command\n");
    }
    else if (strcmp(command, "cd") == 0) {
        printf("Executing CD command\n");
    }
    else if (strcmp(command, "exit") == 0) {
        printf("Executing EXIT command\n");
    }
    else {
        printf("Invalid built-in command\n");
    }
}

int main() {
    char command[50];

    printf("Enter command: ");
    scanf("%49s", command);

    execute_command(command);

    return 0;
}

```

Output

```

spoorthi@LAPTOP:~$ nano builtin_commands.c
spoorthi@LAPTOP:~$ gcc builtin_commands.c -o builtin_commands
spoorthi@LAPTOP:~$ ./builtin_commands
Enter command: pwd
Executing PWD command
spoorthi@LAPTOP:~$ |

```

10(a) — Current Directory and Process Exit

Code

```
#include <stdio.h>
#include <unistd.h>
#include <limits.h>

int main() {
    char path[PATH_MAX];

    if (getcwd(path, sizeof(path)) != NULL) {
        printf("Current Directory: %s\n", path);
    } else {
        perror("getcwd failed");
    }

    printf("Program exiting...\n");

    return 0;
}
```

Output

```
spoorthi@LAPTOP:~$ nano current_directory.c
spoorthi@LAPTOP:~$ gcc current_directory.c -o current_directory
spoorthi@LAPTOP:~$ ./current_directory
Current Directory: /home/spoorthi
Program exiting...
spoorthi@LAPTOP:~$ |
```

10(b) — Export Environment Variable

Code

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    char name[50];
    char value[100];

    printf("Enter variable name: ");
    scanf("%49s", name);

    printf("Enter variable value: ");
    scanf("%99s", value);

    if (setenv(name, value, 1) == 0) {
        printf("Environment variable exported successfully\n");
        printf("%s = %s\n", name, getenv(name));
    } else {
        perror("setenv failed");
    }

    return 0;
}
```

Output

```
spoorthi@LAPTOP:~$ nano export_variable.c
spoorthi@LAPTOP:~$ gcc export_variable.c -o export_variable
spoorthi@LAPTOP:~$ ./export_variable
Enter variable name: MYNAME
Enter variable value: Spoorthi
Environment variable exported successfully
MYNAME = Spoorthi
spoorthi@LAPTOP:~$ |
```